@ codedsoul_05

SEAT INVENTORY v1.0 ★

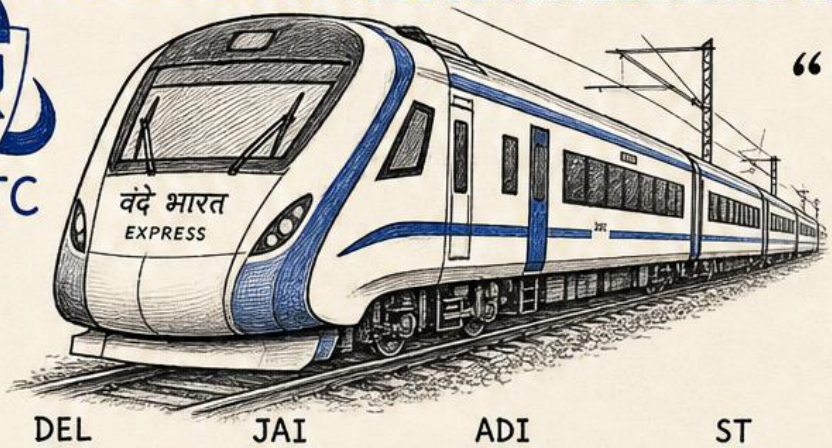
# SEAT INVENTORY

## SEGMENT OVERLAP

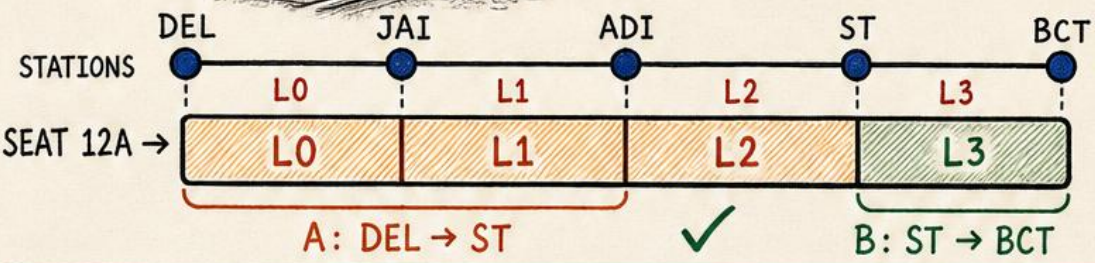
### — HANDWRITTEN DEEP DIVE —

Why One Train Seat Can Be Sold Twice — and Still Be Correct ☺

IRCTC




"One Seat. Two Passengers. Zero Conflict."




Bitmaps • Availability • Fragmentation • Concurrency • Waitlist

### What's Inside ?

1. The Interval Problem
2. Seat as a Bitmap
3. Data Model & Quotas
4. Availability Queries
5. Allocation & Fragmentation
6. Concurrency & Locking
7. Cancellation & Charting
8. RAC & Waitlist Ladder
9. Data Structure Choices
10. Scale, Edge Cases & Recap

+ Cheat Sheet & Interview Q&A

TICKET

### The Core Idea

| | |
|---|---|
| Stations $n$ → Legs $n-1$ | |
| Journey $i \to j$ → legs $i \cdots j-1$ | |
| 0 1 0 1 Free ? | (mask & req) == 0 |
| Book | mask \|= req |
| Release | mask &= ~req |
| Search | MIN(free[legs]) — upper bound only |

Three bit operations run the engine.

### Topics Covered

- ☑ Interval Modelling
- ☑ Occupancy Bitmaps
- ☑ Per-Leg Counters
- ☑ Best-Fit Allocation
- ☑ Atomic Conditional Update
- ☑ Optimistic vs Row Locks
- ☑ Hold → Pay → Confirm
- ☑ Quota Pools
- ☑ RAC & Waitlist
- ☑ Chart Preparation
- ☑ Hot Train Sharding
- ☑ Edge Cases

### Who is this for ?

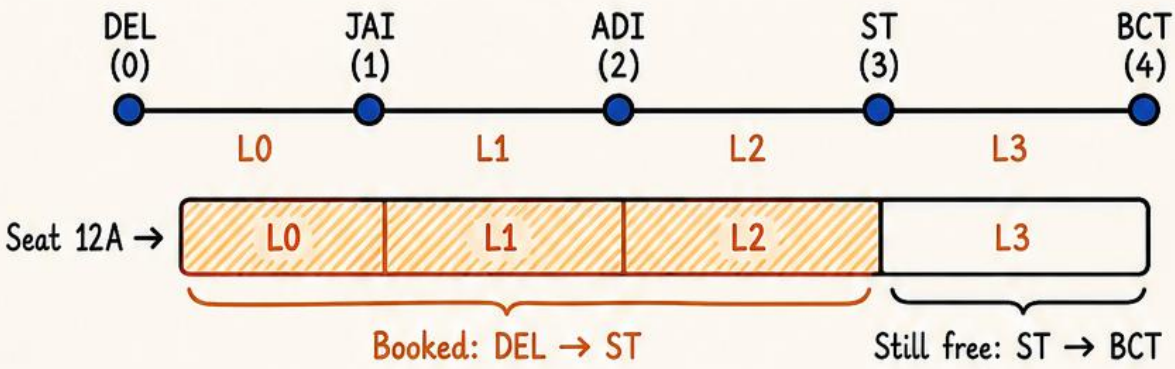
- ★ System Design Interview Preparation
- ★ Backend / Full Stack Developers
- ★ Anyone building booking systems! ☺

"The hardest booking systems are not about counting seats — they are about carving them."

Sell Intervals. Verify Exactly. Pack Deliberately. ★

# 01 — THE PROBLEM

A train seat is not one unit of stock — it is a timeline that can be sold in pieces.




Same seat, same train, two different passengers — both perfectly valid.

## 1. WHY A SIMPLE COUNTER FAILS




- ✗ Counter says "72 seats left", but left for WHICH stretch?
- ✗ Decrement on every booking → you undersell the free legs
- ✗ Do not decrement → you oversell and double-book
- ✗ One number cannot describe availability that varies along the route

## 2. THE CORRECT MENTAL MODEL

- ✓ A route with n stations has n − 1 legs
- ✓ A journey from station i to station j occupies legs i ... j−1
- ✓ A seat is free for that journey only if EVERY leg in the range is free
- ✓ This is an interval / segment problem, not a stock-count problem

## 3. WORKED EXAMPLE

**Booking A:** DEL → ST occupies L0, L1, L2




(Seat 12A)

**Booking B:** ST → BCT occupies L3




(Seat 12A)

Overlap = none → both fit on seat 12A ✓

---

**Booking C:** ADI → BCT occupies L2, L3




(Seat 12A)

Overlap with A at L2 → rejected on seat 12A ✗

## 4. INTERVIEW TIP

Say this sentence early: availability is per leg, not per train.
It instantly shows you understand the real problem.

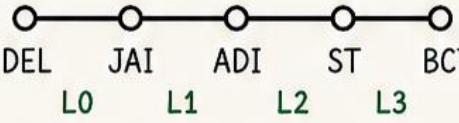
Sell intervals, not units. ★

# 02 — MODELLING A SEAT AS A BITMAP

Give every leg one bit. Now booking becomes bit arithmetic.

## 1. THE ENCODING

Route DEL – JAI – ADI – ST – BCT




Each seat stores an occupancy mask of 4 bits (bit i = leg i is taken).

| L3 | L2 | L1 | L0 |
|---|---|---|---|

## 2. BUILDING A REQUEST MASK

Journey from station index i to j occupies legs i … j−1.

```
mask = ((1 << j) - 1) - ((1 << i) - 1)
```

Examples (bit L0 on the right):

| | | | |
|---|---|---|---|
| DEL → ST | (0 → 3) | = | 0111 |
| ST → BCT | (3 → 4) | = | 1000 |
| ADI → BCT | (2 → 4) | = | 1100 |
| DEL → BCT | (0 → 4) | = | 1111 |

## 3. THE TWO OPERATIONS

**IS IT FREE?**

```
(seat_mask & request_mask) == 0
```

**BOOK IT**

```
seat_mask = seat_mask | request_mask
```

**RELEASE IT**

```
seat_mask = seat_mask & ~request_mask
```

Three bit operations replace an entire availability engine.

## 4. WORKED CHECK

Seat 12A after booking DEL → ST:

seat_mask = 0111

Request ST → BCT = 1000

```
  0 1 1 1
& 1 0 0 0
---------
  0 0 0 0   → FREE ✓
```

new mask = 0111 | 1000 = **1111**

Request ADI → BCT = 1100

```
  0 1 1 1
& 1 1 0 0
---------
  0 1 0 0   → not zero → TAKEN ✗
```

## 5. WHY THIS IS NICE

- ✓ O(1) check and O(1) update per seat
- ✓ Whole seat state fits in one integer for short routes
- ✓ Long routes (60+ stations) → use a byte array or roaring bitmap
- ✓ The same mask works for checking, booking and cancelling

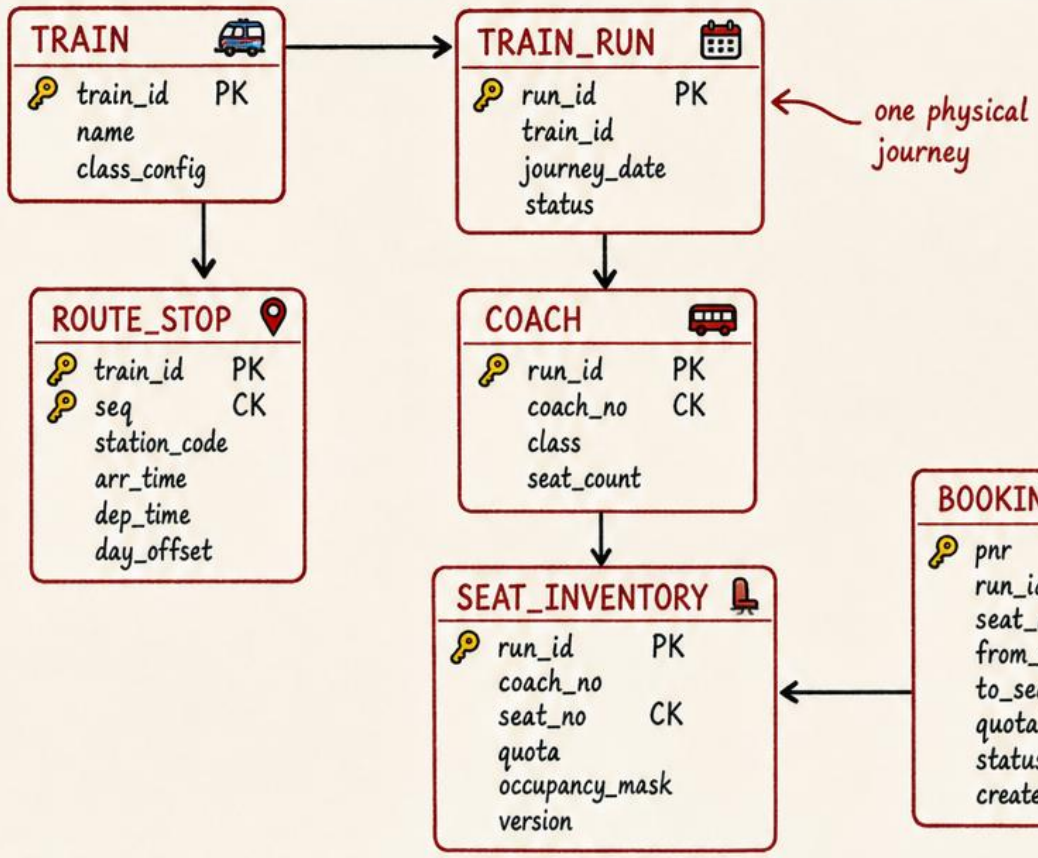
## 6. WATCH OUT

- ✗ Bit count = stations − 1, never stations
- ✗ Index by position in the route, not by station code
- ✗ A train running past midnight is still one journey — index by leg, not by date

One bit per leg turns a hard problem into an AND. ★

# 03 — DATA MODEL, CLASSES & QUOTAS

The unit of inventory is one train on one date — everything hangs off that.




## KEY DECISIONS

- ✓ run_id = (train_id, journey_date) — the natural inventory boundary
- ✓ Store from_seq / to_seq on the booking, not station codes
- ✓ day_offset on route stops handles trains crossing midnight
- ✓ version column enables optimistic concurrency
- ✓ Every booking is reproducible from (seat, from_seq, to_seq)

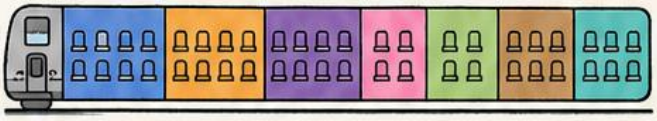
## CLASSES ARE SEPARATE POOLS

Sleeper, 3AC, 2AC, 1AC each have their own coaches and seats.

NOTE: A 3AC request can never consume Sleeper inventory — model class as a hard partition.

## QUOTAS ARE SUB-POOLS

General · Tatkal · Premium Tatkal · Ladies · Senior Citizen · Defence · Handicapped



RULES:

- Each quota owns a fixed slice of seats
- A quota can be released back to general at chart time
- Availability must be computed per (class, quota, leg range)

NOTE: Quota multiplies your availability queries — cache them separately.

## STORAGE CHOICES

- PostgreSQL / MySQL — bookings and seat inventory (needs transactions)
- Redis — cached availability counts, hot train state
- Kafka — booking and cancellation event stream
- Archive — completed runs moved out after the journey date

NOTE: Inventory must be transactional. Availability display can be cached.

Model the train-date first; every other decision follows from it. ★

# 04 — AVAILABILITY QUERIES

Millions of searches, far fewer bookings — so search must be cheap and booking must be exact.

## EXACT METHOD (source of truth)

Given run_id, class, quota, from_seq i, to_seq j:

```
req = mask(i, j)
available = COUNT(seats WHERE
            occupancy_mask & req = 0)
```

- Always correct
- Cost = O(number of seats in that class)
- Used at booking time, and for the final confirm

## FAST METHOD (for display)

Keep a per-leg free counter:

```
free[L0], free[L1] ... free[Ln-1]
```

```
estimate = MIN(free[i ... j-1])
```

- O(number of legs), tiny and cacheable
- Updated incrementally on every booking and cancellation

## THE TRAP — MIN IS ONLY AN UPPER BOUND

Two seats, two legs:

Seat A: free on L0, busy on L1

Seat B: busy on L0, free on L1

free[L0] = 1, free[L1] = 1 → MIN = 1

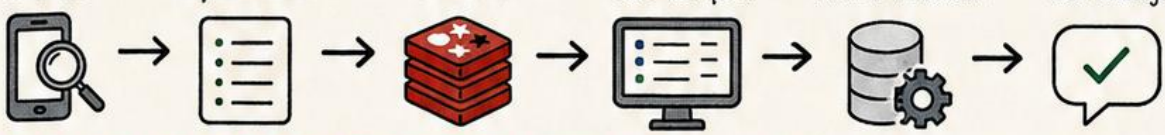
But NO single seat is free across both legs → true answer is 0

| | L0 | L1 |
|---|---|---|
| Seat A | ✓ | ✗ |
| Seat B | ✗ | ✓ |

Use MIN for the search page.
Re-verify exactly before you take money.

## SEARCH REQUEST PATH

1. User searches DEL → BCT on a date
2. Look up run_id and station sequence numbers
3. Read cached per-leg counters from Redis
4. Show approximate availability per class and quota
5. On booking, recompute exactly inside a transaction
6. Confirm or fail fast with a clear message



## CACHING RULES

- ✓ Cache per (run_id, class, quota, from_seq, to_seq) with a short TTL
- ✓ Invalidate on every booking or cancellation for that run
- ✓ Hot trains get their own in-memory copy
- ✓ Never book directly from a cached number

## COUNTER-EXAMPLE (TWO SEATS, TWO LEGS)

| Seat | L0 | L1 | Across L0–L1? |
|---|---|---|---|
| A | ✓ (free) | ✗ (busy) | ✗ |
| B | ✗ (busy) | ✓ (free) | ✗ |
| free[L0] = 1 | | free[L1] = 1 | True answer = 0 |

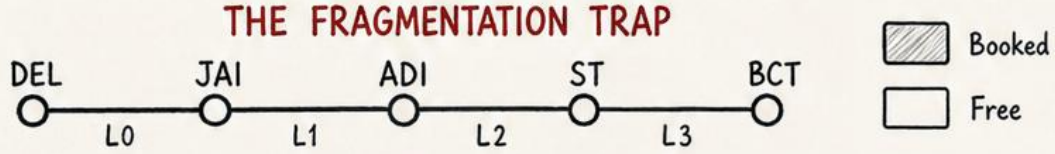
Note: A slightly stale search page is fine. A stale booking is a double-booked passenger.

★ Approximate to browse. Verify to sell. ★

# 05 — ALLOCATION & FRAGMENTATION

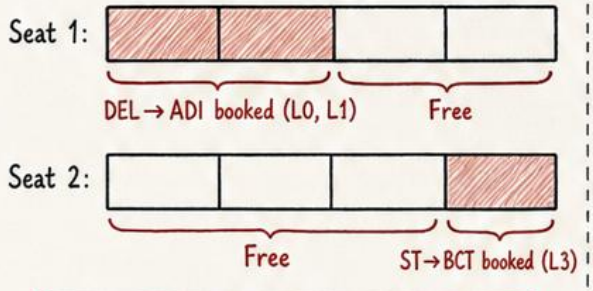
Which free seat you pick decides how much you can still sell later.

## THE FRAGMENTATION TRAP




### BAD PLACEMENT (FRAGMENTS)

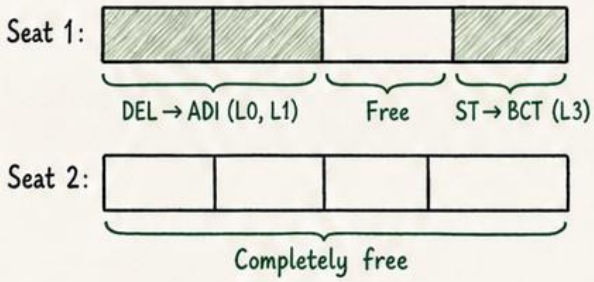
Route: DEL → JAI → ADI → ST → BCT
Legs: L0 L1 L2 L3




Now a DEL → BCT request needs all 4 legs on one seat → REJECTED ✗

### GOOD PLACEMENT (PRESERVES CAPACITY)

Route: DEL → JAI → ADI → ST → BCT
Legs: L0 L1 L2 L3




Now a DEL → BCT request fits on Seat 2 (all 4 legs free) → ACCEPTED ✓

Same two bookings. Different seat choice. Completely different remaining capacity.

## ALLOCATION STRATEGIES

- First fit — take the first free seat. Fast, fragments badly.
- Best fit — prefer a seat where the new block sits ADJACENT to an existing booking. Keeps long free runs intact.
- Worst fit — spread bookings out. Almost always wrong here.

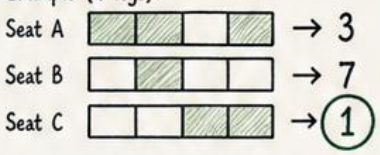
Best fit is the right default — it is the same idea as memory allocation.

## SCORING A CANDIDATE SEAT

Prefer the seat that:

1. Has the fewest free legs left after this booking
2. Touches an existing booking on either side (no 1-leg holes)
3. Keeps the longest contiguous free run on OTHER seats

Example (4 legs):




Lower score is better. (Best circled)

## REAL-WORLD PREFERENCES

(they fight fragmentation)

- Berth preference: lower, middle, upper, side
- Group bookings should sit together
- Ladies and senior-citizen quota berth rules
- Coach balance for weight distribution

Passenger preference and packing efficiency pull in opposite directions — resolve it explicitly.

## PRACTICAL COMPROMISE

- ✓ Filter candidates by preference first
- ✓ Then rank the survivors by fragmentation score
- ✓ Reserve a few whole seats for full-route journeys on long-distance trains
- ✓ Re-optimise at chart preparation, when the picture is final



Keep some whole seats unfragmented for full-route travellers.

★ Packing seats is bin packing — place deliberately, not first-come. ★

# 06 — CONCURRENCY & DOUBLE BOOKING

Two users, one seat, overlapping legs, same millisecond.

## THE RACE

Timeline

| User A | | User B |
|---|---|---|
| A reads seat 12A mask = 0000 | t1 | (no action) |
| (no action) | t2 | B reads seat 12A mask = 0000 |
| A checks 0000 & 0111 = 0 → free | t3 | (no action) |
| (no action) | t4 | B checks 0000 & 0110 = 0 → free |
| A writes mask = 0111 | t5 | (no action) |
| (no action) | t6 | B writes mask = 0110 ← A's booking is silently erased |

Read-then-write is always a bug under concurrency.

## FIX 1 — ATOMIC CONDITIONAL UPDATE (best)

```
UPDATE seat_inventory
SET    occupancy_mask = occupancy_mask | :req,
       version = version + 1
WHERE  run_id = :run
  AND  seat_no = :seat
  AND  (occupancy_mask & :req) = 0;
-- rows affected = 1 → you won the seat
-- rows affected = 0 → someone else took it,
                       try the next seat
```

Test and set in a single statement.
No lock held across a network round trip.

## FIX 2 — PESSIMISTIC ROW LOCK

```
BEGIN;
  SELECT occupancy_mask FROM seat_inventory
   WHERE run_id = :run AND seat_no = :seat
    FOR UPDATE;
  -- check, then update
COMMIT;
```

- Simple to reason about
- Holds a lock for the whole transaction
- Under Tatkal load, lock waits become the bottleneck

## TRANSACTION BOUNDARY

- ☑ One transaction must cover:
  seat mask update + booking row insert + PNR generation
- ☑ If any part fails → roll back everything

Note: Never call the payment gateway inside this transaction.

## BOOKING SAGA (money + seat)

HOLD TIMER



02:30
(counting down)

1. Hold the seat with a short TTL (a few minutes)
2. Take payment outside the transaction
3. Payment success → confirm the booking
4. Payment failure or timeout → release the hold, auto-refund
5. Idempotency key makes every retry safe

## DEADLOCK AVOIDANCE




Always lock seats in ascending seat_no order



Keep transactions short



Retry on deadlock with backoff

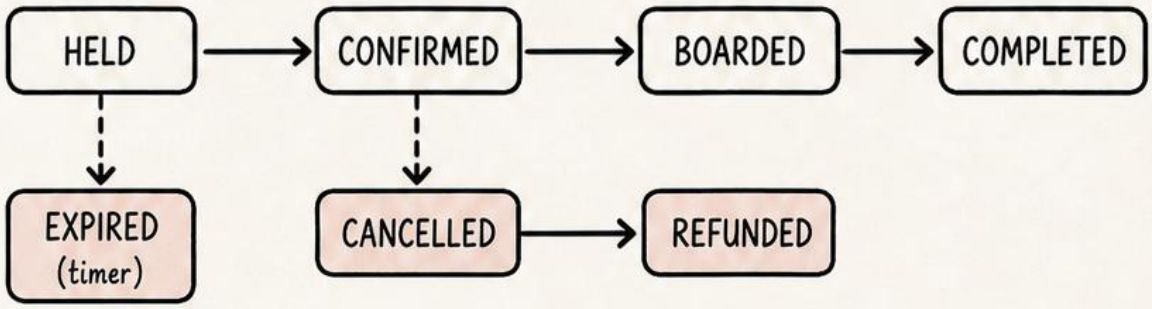


Never lock the whole coach when one row will do

Let the database decide the winner — in one statement. ★

# 07 — CANCELLATION, RELEASE & CHARTING

*Freeing a segment is easy. Deciding who gets it next is the hard part.*




Every transition also mutates the seat's occupancy mask.

## RELEASING A SEGMENT

```
occupancy_mask = occupancy_mask & ~req
```

Example: mask 1111, cancelling DEL → ST (0111)

```
  1111   (current mask)
& 1000   (~0111)
------
  1000   (result)
```

1111 & 1000 = 1000 → legs L0,L1,L2 free again ✓

Release exactly the legs that booking held — never clear the whole mask.

## PARTIAL CANCELLATION

- One passenger of four cancels → free that seat only
- Boarding point changed to a later station → release the earlier legs
- Journey shortened → release the tail legs

Each of these is a mask update, not a delete.

## WHO GETS THE FREED SEGMENT?

1. Waitlisted passengers whose journey fits the freed legs
2. RAC passengers upgrading to a full berth
3. Return to the general availability pool

Process in order of waitlist position — fairness is auditable and disputed often.

## CHART PREPARATION

A few hours before departure:

- Unused quota seats are released to the general pool
- Waitlist is cleared as far as capacity allows
- RAC passengers are upgraded where possible
- Seats are re-optimised to reduce fragmentation
- After charting, inventory becomes largely frozen (current booking only)




## CORRECTNESS RULES

- ✓ Cancellation and re-allocation happen in one transaction
- ✓ Every mask change is written to an event log for audit
- ✓ Refund is triggered only after the seat is released
- ✓ Re-allocation must be idempotent — the job may run twice

A cancellation is not a delete — it is an interval being returned.

# 08 — RAC & WAITLIST AS EXTENDED INVENTORY

Capacity does not stop at the last berth — it continues as RAC, then as a queue.

1. **CONFIRMED** — full berth, one passenger per segment ↑ promotes on cancellation
2. **RAC** — side berth shared by two passengers, seat guaranteed ↑ promotes on cancellation
3. **WAITLIST** — no seat yet, position in a queue

## WHY RAC EXISTS

- Absorbs expected cancellations without leaving berths empty
- Guarantees the passenger travels, even if seating is shared
- Sized from historical cancellation rates for that train and route

Note: RAC is a deliberate, controlled overbooking layer.

## WAITLIST TYPES

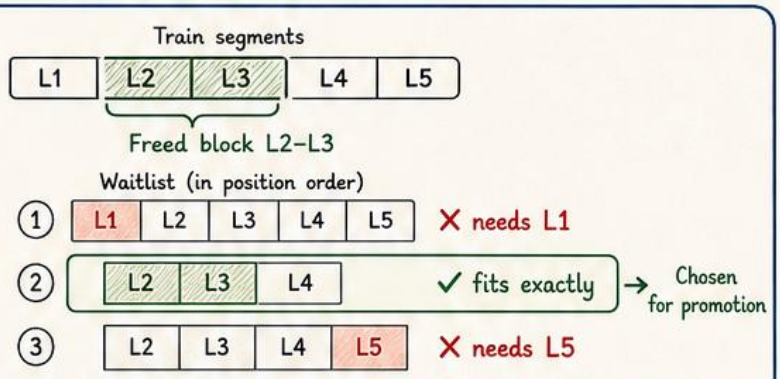
- GNWL — general, boarding at an originating station, best chance
- RLWL — remote location, confirms only from that station's own quota
- PQWL — pooled across several intermediate stations
- TQWL — Tatkal waitlist, clears last and rarely

RLWL and PQWL cannot consume another station's freed segment — the quota decides eligibility.

## SEGMENT-AWARE PROMOTION

When legs L2–L3 are freed:

1. Scan the waitlist in position order
2. Skip anyone whose journey needs legs outside L2–L3
3. Promote the first passenger who fits exactly
4. Update the mask, generate the confirmed berth, notify




Note: A freed short segment can confirm a short journey while a longer one still waits — this surprises passengers, so it must be explainable.

## PROMOTION SAFETY

- ✓ Run promotion as one transaction per promotion
- ✓ Make it idempotent — the job can rerun after a crash
- ✓ Log the reason a passenger was skipped (for support tickets)
- ✓ Never promote after charting except through current booking

## CAPACITY MATH (illustrative)

72 confirmed berths + 21 RAC + waitlist cap ≈ 100

Note: Waitlist is capped so you never sell hope indefinitely.

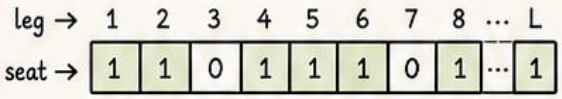
★ Confirmed, RAC and waitlist are one continuous capacity ladder. ★

# 09 — DATA STRUCTURE CHOICES

Four ways to model an interval. Pick by route length and query mix.

## 1. PER-SEAT BITMAP

Store one mask per seat.



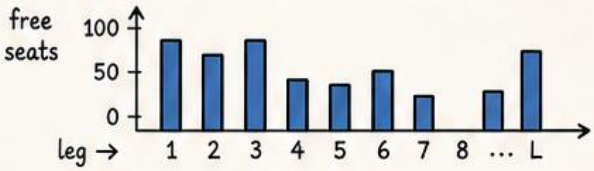

Check O(1), book O(1),
memory = seats × legs bits

- ✓ Simple, exact, cache friendly
- ✗ Availability count is O(seats) — must scan

**Best for: normal routes (up to ~64 legs fits in one integer)**

## 2. PER-LEG COUNTERS

Store free[leg] for the whole class.



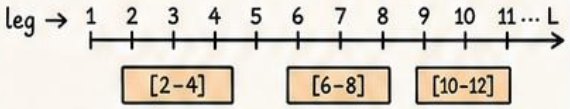

Availability estimate O(legs)

- ✓ Tiny, perfect for search pages and caching
- ✗ MIN across legs is only an upper bound — cannot allocate a seat from it

**Best for: display and search, never for the final booking**

## 3. INTERVAL LIST PER SEAT

Store the booked intervals as a sorted list.



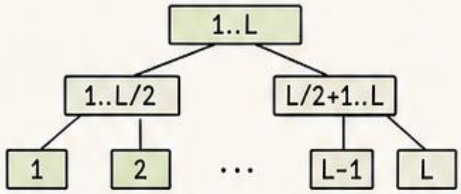

Check = overlap scan O(k)
where k = bookings on that seat

- ✓ Natural for very long routes and sparse bookings
- ✓ Easy to render "who occupies which stretch)
- ✗ Slower than bit operations, needs merging on release

**Best for: very long routes and sparse bookings**

## 4. SEGMENT TREE / FENWICK

Tree over legs storing max occupancy in a range.




Range query and range update in O(log n)

- ✓ Excellent when you need "max booked across a range" quickly
- ✗ Overkill for 20–60 legs; more code, more bugs

**Best for: very long routes or aggregated analytics**

| Structure | Check | Book | Count avail | Use |
|---|---|---|---|---|
| Bitmap | O(1) | O(1) | O(seats) | Booking |
| Leg counters | O(L) | O(L) | O(L) approx | Search |
| Interval list | O(k) | O(k) | O(seats·k) | Long routes |
| Segment tree | O(logL) | O(logL) | O(logL) | Rare/large |



### THE PRACTICAL ANSWER

Use both: per-leg counters in Redis for fast search, per-seat bitmaps in the database as the source of truth. Reconcile the counters from the bitmaps periodically.

Choose the structure from the query you run most. ★

# 10 — SCALE, EDGE CASES & RECAP

## BOOKING PATH — 8 STEPS

| ① | ② | ③ | ④ | ⑤ | ⑥ | ⑦ | ⑧ |
|---|---|---|---|---|---|---|---|
| Search | Cached availability | Select class & quota | Build request mask | Pick seat (best fit) | Atomic conditional update | Payment | Confirm PNR |

## SHARDING & HOT TRAINS

- Shard by run_id = (train_id, journey_date) — all seats for one journey live together
- A transaction never crosses shards
- Tatkal at 10:00 AM makes a few run_ids extremely hot

**Mitigations:**

- ✅ Queue requests per run_id and process them in order (single-writer per train)
- ✅ Keep that run's inventory in memory with a write-ahead log
- ✅ Admission control: accept only as many requests as seats plus a margin
- ✅ Virtual waiting room instead of letting everyone hammer the database

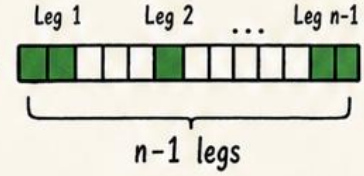
## EDGE CASES

- Train crossing midnight → use day_offset, not calendar date
- Route with a reversal or repeated station → index by sequence, never by station code
- Diverted or short-terminated train → truncate legs and re-issue
- Boarding point change → release the earlier legs
- Group booking → all seats must succeed together, or none
- Different classes are separate pools — never mix

## WHAT AN INTERVIEWER LISTENS FOR

- "Availability is per leg, not per train."
- "A seat is free only if every leg in the range is free."
- "MIN across legs is an upper bound, not the answer."
- "Best-fit placement prevents fragmentation."
- "One conditional UPDATE decides the winner atomically."
- "Hold the seat, then take payment — never the reverse."

## 30-SECOND SUMMARY




A route with n stations has n−1 legs. Every seat carries an occupancy bitmap over those legs, so a journey fits only when (seat_mask AND request_mask) is zero. Search uses cached per-leg counters for speed, but booking re-verifies exactly and claims the seat with a single conditional update. Seats are chosen by best fit to avoid fragmentation, cancellations return intervals rather than deleting rows, and freed segments flow back down the RAC and waitlist ladder.

**Sell Intervals. • Verify Exactly. • Pack Deliberately.**

# DEEP DIVE COMPLETE ✓